

A Self-Modifying Predictor: Evolving Architecture, Hyper-Parameters, and Structure to a Target Error

Vasili Gavrilov*

2026

Abstract

Ordinarily a neural network must be handed its shape before it can learn: how many layers, how wide, which activation, and so on. The system studied here is handed only the training data and a target error, and finds the rest itself by evolution: its depth and width, whether each layer is fully connected or a weight-sharing local one, and its own learning rate, momentum, and activation, training until it reaches the target. It turns the usual **predict(topology, activation, ...)** into **self_modifying_predict(data, error)**. Beyond an earlier fixed-budget study, three things are new: training hyper-parameters that co-evolve with the topology under a self-adaptive mutation rate; a structural mutation between a dense and a weight-shared *convolution-like* layer, so the search can rediscover LeCun’s local inductive bias on its own [3] (its back-propagation verified against finite differences); and *error control*, which replaces “whose error is lower at a fixed budget” with *time-to-target*, the work each method needs to reach the same error. We test one hypothesis: given a target error, does the self-modifying search reach it with less work than random? Over 400 seeds it does: reaching the target more often (270 versus 251; paired McNemar $p \approx 0.02$) and, when both reach it, sooner on more runs than not (head-to-head 146 to 87; sign test $p \approx 7 \times 10^{-5}$). The edge is modest ($\approx 63\%$ of decided runs) but statistically clear, and points *opposite* to the fixed-budget parity, where random search remains hard to beat.

1 Introduction

Neural architecture search (NAS) automates a choice a human normally makes: the shape of the network: how many layers, how wide, how connected. Modern NAS methods differ mainly in their *search strategy*: reinforcement learning [7], gradient-based relaxation [10], Bayesian optimization, random search, and evolutionary search [12]. The evolutionary branch is the subject here, in its *evolutionary NAS* form [8, 9]: the topology is evolved while each candidate’s weights are trained by gradient descent (as opposed to neuroevolution such as NEAT [5], which evolves weights and topology together without gradients).

Two signatures frame the project. The first is the ordinary one, in which the architecture and training knobs are arguments the user must supply:

```
predict(Train, Test, Topology, Activation, Jitter, ...).
```

The second is the one this system aims at, in which the user supplies only the data and the error they are willing to accept:

```
self_modifying_predict(Train, Test, Error).
```

*Independent researcher. Physics and computer science by education; a professional software engineer with a background in numerical optimization and machine learning, including rule engines and expert systems. Reference implementation: <https://github.com/Anode1/SMBPANN>.

Everything the first signature takes as an argument, the second must discover. The engine grew out of the author’s 1997 seminar thesis, which derived the general multi-layer delta rule the training still uses [2]; an earlier version of this study [13] took the first step toward the second signature (evolving layer counts and widths) but still compared methods the ordinary way: fix a compute budget, run, and see whose final error is lower. This paper closes the gap to the second signature in three moves (co-evolved hyper-parameters, an evolvable convolution-like layer, and error control) and, with error control in place, asks the question the second signature actually poses to a user: not “whose error is lower” but “who reaches *my* error with less work.”

A recurring, humbling result in this field is that *random search is hard to beat*. Bergstra and Bengio [6] showed plain random search is a strong baseline for hyper-parameter optimization; Li and Talwalkar [11] showed random search (with weight sharing) is competitive with leading NAS methods, and documented a reproducibility problem in the field along the way. Our contribution is not a new method. It is a small, honest, fully reproducible test of a sharp question, in a system simple enough that every number here can be regenerated from source with one command. The self-adaptive machinery we lean on is standard in the optimization literature the author has worked in, the evolution strategies of Rechenberg and Schwefel [4], in which the step size is itself under selection.

2 Hypothesis

We state the claim to be tested, and its null, plainly.

H. Given only a training set, a test set, and a target validation error, a self-modifying evolutionary search, one that co-evolves architecture and training hyper-parameters and can grow structural (convolution-like) layers, reaches the target using *fewer* candidate evaluations than a matched random search that draws the same kind of random architectures.

H₀. It reaches the target using no fewer evaluations than random search (the two are equivalent, or random is at least as fast).

The measure is *time-to-target*: the number of generations (equivalently, evaluations, since each generation is a fixed batch of them) a method needs before its best validation error first drops to the target. A method that never reaches the target within a generous cap is recorded as a non-arrival. This is the quantity a user of `self_modifying_predict` cares about, and it is the quantity the earlier fixed-budget protocol could not measure.

3 Method

3.1 Network and training

A network is a feed-forward stack of dense and convolution-like layers. For a dense layer ℓ with weight matrix $W^{(\ell)}$ and bias $b^{(\ell)}$, the forward pass is

$$z^{(\ell)} = W^{(\ell)}a^{(\ell-1)} + b^{(\ell)}, \quad a^{(\ell)} = g(z^{(\ell)}), \quad (1)$$

with g a per-network hidden activation (logistic sigmoid $g(z) = 1/(1 + e^{-z})$, tanh, or ReLU) chosen by the genome; the output layer is always a dense sigmoid. Weights are trained by back-propagation (the generalized delta rule) with momentum: with local error $\beta_i = (d_i - a_i)g'(z_i)$ at output units and $\beta_i = g'(z_i) \sum_k W_{ki}^{(\ell+1)}\beta_k^{(\ell+1)}$ at hidden units, the update is

$$\Delta W^{(\ell)}(t) = r \beta^{(\ell)}(a^{(\ell-1)})^\top + \alpha \Delta W^{(\ell)}(t-1), \quad (2)$$

with learning rate r and momentum α , both carried in the genome. This is the generalized multi-layer delta rule, momentum, and sigmoid the author derived in a 1997 seminar thesis [2]; the engine executes exactly that mathematics. Weights are initialized uniformly in $[-2.4/f_i, 2.4/f_i]$, where f_i is the fan-in (also from the thesis). Training is online (one example at a time), so a worker’s memory footprint is bounded regardless of dataset size. All arithmetic uses 32-bit floats; all randomness flows through a seeded 32-bit xorshift generator, so the same seed gives the same result on any machine.

3.2 A convolution-like layer as an evolvable structure

A hidden layer may instead be *convolutional* in one dimension: F filters, each a shared kernel of width K , swept across the previous layer’s $d_{\ell-1}$ units. A filter f at position p computes

$$z_{f,p}^{(\ell)} = b_f + \sum_{k=0}^{K-1} w_{f,k} a_{p+k}^{(\ell-1)}, \quad p = 0, \dots, d_{\ell-1} - K, \quad (3)$$

so the layer’s width is *derived*, $d_\ell = F(d_{\ell-1} - K + 1)$, not a free parameter, and its weight count is only FK . This is the pair of properties LeCun introduced for images (local receptive fields and weight sharing [3]), here made a *move* the search can take rather than a design the user imposes. Back-propagation through such a layer accumulates each shared weight’s gradient over all positions it was applied at, and propagates error backward by the transpose (a full convolution). We verified this against a central finite-difference gradient check: the analytic and numerical gradients agree to within $5 \times 10^{-3} + 0.02|g|$ on every weight, so the structural move trains correctly.

3.3 Genome, co-evolution, and self-adaptation

A *genome* carries everything the second signature must discover:

- the topology: for each hidden layer, a kind (dense or convolutional) and its free parameters (a dense width, or a filter count F and kernel width K);
- the training hyper-parameters: learning rate r , momentum α , and hidden activation;
- a self-adaptive mutation rate σ .

The input and output widths are fixed by the problem. One *mutation* makes a single small change: perturb a hidden layer’s free parameter by one; flip a hidden layer between dense and convolutional; or insert or delete one hidden layer (of a random kind), within bounds on depth ($L_{\max}$) and width ($W_{\max}$). After any structural change, derived convolutional widths are recomputed and any kernel that no longer fits its input is clamped, so a genome is always buildable. There is no crossover: recombining two arbitrary topologies is ill-posed (the competing-conventions problem), and mutation-only evolution is the current default in evolutionary NAS [9].

Reproduction is *self-adaptive* in the sense of evolution strategies [4]. An offspring inherits its parent’s mutation rate, perturbs it log-normally, and applies that many topology mutations:

$$\sigma' = \sigma \cdot \exp(\tau \mathcal{N}(0, 1)), \quad \text{number of mutations} = \text{round}(\sigma'), \quad (4)$$

clamped to $[\sigma_{\min}, \sigma_{\max}]$, with $\tau = 0.35$. The training hyper-parameters co-evolve the same way, by gentler log-normal steps on r and α (each clamped to a sane range) and an occasional switch of activation. Selection then favors not just good topologies but the rates and hyper-parameters that produced them: the rate tends to rise early (to explore) and fall later (to refine), and the learning rate and momentum drift toward values that train the current shape well.

Algorithm 1 Self-modifying search under error control, with matched random control

```
1: population  $\leftarrow P$  random genomes; both bests  $\leftarrow \infty$ 
2: for generation = 1 to  $G$  (safety cap) do
3:   evaluate the population in parallel (one worker process per genome)
4:   keep the best  $k$  (elite); update the genetic best-so-far
5:   refill: each new genome reproduces self-adaptively from a random elite (topology, hyper-
     parameters, and rate all co-evolve)
6:   control: evaluate  $P$  fresh random genomes; update control best-so-far
7:   record the first generation each best reaches the target error
8:   stop once both bests are at or below the target
9: end for
10: return generations-to-target for the genetic algorithm and for the control
```

3.4 Evolutionary search, error control, and the random control

The search keeps a population of P genomes, retaining the best k (elitism) and refilling the rest by self-adaptive reproduction from the elite (Algorithm 1). The *matched random-search control* draws P fresh random genomes every generation and keeps its own best-so-far, so per generation it makes exactly the same number of evaluations as the genetic algorithm. The genetic algorithm’s *initial* population is drawn from the same random distribution as the control, so both start from the same place and differ only in what they do next. Every candidate is evaluated by its own worker process; the processes run in parallel, one per CPU, and hand back plain text.

Under *error control*, both methods run until their best validation error first reaches the target, with a generation cap G only as a safety stop. Because the fitness a candidate is scored by *is* its validation error, the target is a single number that plays both roles: the objective each candidate minimizes and the condition that halts the search. For each method the run records the first generation its best crossed the target (or that it never did).

3.5 The task

To ask whether architecture matters, the task must be one where it does, and which the networks can actually learn, so that better architectures score better. We generate a synthetic classification task, reproducibly, from a seed. Inputs x are uniform in $[-1, 1]^d$; the label is

$$y = [\sin(f w \cdot x) > 0], \quad (5)$$

for a fixed random projection w and frequency f , with a fraction of labels flipped as noise. Higher f makes more decision stripes and demands more capacity. The projection $w \cdot x$ is a spatial mixing of adjacent inputs, so a local, weight-sharing layer has structure to exploit, which is why the convolution-like move is not idle here. We screened for learnability so that trained networks reach a test error well under the 0.25 a constant 0.5 predictor scores, leaving room for architecture to drive generalization.

3.6 Protocol and metrics

Each *run* uses a task and a search seed; within a run the genetic algorithm and the control see the same task and the same 80/20 train/test split, and are trained and initialized identically (a fixed worker seed), so the two differ only in which genomes they evaluate. Because the comparison is paired on the task this way, the head-to-head already controls for task difficulty. Two task designs are used: a *fresh task per run* (task seed = run index), which samples the advantage over the task distribution (the main test), and a *fixed task* with only the search seed varying, repeated over several fixed tasks, which exposes how the advantage varies from one landscape to

Table 1: Self-adaptive versus fixed mutation, small space, 10 seeds. Entries are genetic wins out of 10 (mean test-MSE gap over random). Self-adaptation wins more often and by a wider margin at every starting rate.

starting rate ($-M$)	fixed	self-adaptive
1	5 (+0.0021)	8 (+0.0026)
3	5 (+0.0004)	6 (+0.0017)
6	5 (+0.0009)	7 (+0.0023)

another (its heterogeneity). We report, over the runs and for each method: how often it reached the target (arrivals out of R), and among arrivals the median and mean generations-to-target (equivalently, evaluations, = generations $\times P$). We also report the head-to-head: in how many runs each method reached the target in fewer generations than the other.

4 Experiments

4.1 Correctness of the structural move

Before any search, the convolution-like layer is checked in the unit suite: a forward pass with known shared weights reproduces the hand-computed sums (confirming weight sharing), and the analytic gradient of every weight matches a central finite-difference estimate to within $5 \times 10^{-3} + 0.02 |g|$. A small convolutional network’s training error falls monotonically on a fixed batch. Only then is the move handed to evolution.

4.2 Self-adaptive versus fixed mutation (controlled)

We first isolate the one clean A/B from the earlier study: a fixed mutation rate against the same rate left free to self-adapt, in a small search space ($L_{\max} = 3$, $W_{\max} = 16$), holding everything else equal ($d = 4$, 600 samples, $f = 3$; $P = 10$, $G = 8$; 1200 training epochs; 10 seeds). Only the adaptation is toggled.

Table 1 shows the effect: a fixed rate is about a coin flip against random (5 of 10) whatever the rate, and self-adaptation lifts this to 6–8 of 10 and widens the gap. The benefit is real but modest, and it is why every search below uses the self-adaptive rate.

4.3 Time-to-target: the main test

The central experiment runs the self-modifying search under error control against its matched random control and measures time-to-target. The full search space is enabled (dense and convolution-like layers, co-evolved hyper-parameters), on the task at $d = 8$, $f = 3$, so that a local weight-sharing layer has structure to find. Settings: $P = 10$, elitism $k = 2$, $L_{\max} = 3$, $W_{\max} = 16$, initial mutation rate 2; 1000 training epochs; 300 samples with 2% label noise and an 80/20 train/test split; crossover off; target error 0.17; a safety cap of $G = 30$ generations; a fresh task per run over $R = 400$ seeds. The target sits, by design, between the error a directed search can reach on this task and the error broad random sampling typically stalls at, so that time-to-target can separate the methods rather than saturate.

Table 2 reports the race. The self-modifying search reaches the target on more seeds (270 versus 251 of 400); because the seeds are paired (each run gives both methods the same task), this is a McNemar comparison on the 59 runs where exactly one method arrived (39 for the genetic algorithm against 20 for random), which is significant at $p \approx 0.02$. When both methods reach the target, the genetic algorithm gets there in fewer generations on more runs than not: the per-run head-to-head is 146 to 87 (with 57 ties and 110 runs on which neither arrived within

Table 2: Time-to-target over $R = 400$ seeds (fresh task per run), target error 0.17, safety cap 30 generations, $P = 10$ (so evaluations = $10 \times$ generations). Means and medians are over the runs that reached the target; “head-to-head wins” counts the runs in which that method reached the target in strictly fewer generations than the other (with 57 ties and 110 runs on which neither reached it). The self-modifying search reaches the target more often, and wins the paired race on more runs than random does.

method	reached target	mean gens	median gens	head-to-head wins
self-modifying GA	270 / 400	5.8	3	146
random search	251 / 400	6.1	3	87

Table 3: Fixed-task heterogeneity: per-task head-to-head over 50 search seeds each, 8 tasks. “GA” and “RAND” count the search seeds on which that method reached the target 0.17 in strictly fewer generations; “neither” counts seeds where neither did within the 30-generation cap. *Populated from the `scripts/error_test_multi.sh` run in progress.*

task seed	GA sooner	RAND sooner	tie	neither
1–8 (per task)	–	–	–	–
pooled (400 runs)	–	–	–	–

the cap), and a one-sided sign test on the 233 decisive runs gives $p \approx 7 \times 10^{-5}$. The typical times are close (median 3 generations for both, mean 5.8 versus 6.1): the effect is not that the winner is far faster, but that the genetic algorithm is the faster of the two on about 63% of the runs that are decided, consistently enough over 400 seeds to be clear.

The direction is the point. Under the earlier fixed-budget measure the same kind of search sat at parity with random or below it; under time-to-target it comes out ahead. The two measures are not in conflict. Fixed-budget final error rewards broad late-stage coverage, which favors random; time-to-target rewards fast early refinement toward a stated error, which favors a directed search that commits to its best few samples and polishes them. The interface `self_modifying_predict` poses the second question, not the first, and by that question the self-modifying search beats random search here, modestly in size, but clearly.

4.4 Fixed-task heterogeneity

The aggregate edge above averages over 400 different tasks. To see how it is composed, we hold the task fixed and vary only the search seed, repeated over several tasks. We fix each of 8 tasks (task seeds 1 to 8) in turn and race the two methods over 50 search seeds each (400 runs total), at the same settings as the main test ($d = 8$, $f = 3$, 2% noise, 300 samples, 1000 epochs; $P = 10$, $k = 2$, $L_{\max} = 3$, $W_{\max} = 16$, initial mutation rate 2; target 0.17, cap 30; crossover off). For each fixed task we count, over its 50 search seeds, how often each method reached the target sooner (the per-run head-to-head).

Table 3 reports the spread. Two things are expected, and are already visible in the early data: the search seed genuinely matters (arrival generations spread widely even on one fixed task), and the winner depends on the task, some landscapes favoring the directed search and some favoring random. The pooled count recovers the aggregate direction of the main test. The advantage is therefore a distribution over tasks, not a uniform effect, which is the honest qualification of the headline result.

4.5 Fixed-budget parity, for context

For continuity with the earlier study we note its fixed-budget finding, which the new machinery does not overturn: when the search space is enlarged and the budget held fixed, the genetic algorithm hovers near parity with random search and does not reliably beat it, consistent with [6, 11]. Error control does not change that landscape; it changes the *question*, from final error at a fixed budget to work-to-a-fixed-error, which is the one the target-driven interface poses.

4.6 Result

In one line: under time-to-target and over 400 paired seeds, the self-modifying search reaches a stated target error more often than random search (270 versus 251, $p \approx 0.02$) and sooner on the majority of the runs both solve (head-to-head 146 to 87, $p \approx 7 \times 10^{-5}$), a modest but statistically clear edge that reverses the fixed-budget parity. Every candidate is trained by back-propagation, the convolutional gradient is verified against finite differences, and each figure is regenerable from the source with a single command.

5 Discussion

The structural move is now available to evolution, and it trains correctly: the search can, in principle, assemble local weight-sharing layers rather than being told to. Whether it *should*, on a given task, is exactly what time-to-target measures, and measures more faithfully than fixed-budget final error, because a user of `self_modifying_predict` specifies an error and pays for the work to reach it. Two forces set that work. Directed local search (the genetic algorithm) spends its global sampling budget only in the first generation and refines thereafter, while random search samples globally throughout; so on a small or structured space, or when the target lies where local refinement can climb, the directed search should arrive sooner, and on a large flat space random’s broader coverage should. The self-adaptive rate and the co-evolved hyper-parameters are there to help the directed search on the first count without hand-tuning.

Limitations. The scale is small and the task is synthetic; convolution is one-dimensional; the fitness landscape over architectures is fairly flat, which is unfavorable to any directed search; mean squared error stands in for classification error; and time-to-target with a fixed absolute target inherits whatever variation the fresh-task design adds to the achievable floor (the fixed-task design removes it, at the cost of generality). None of this is hidden, and each bounds how far to read the result.

6 Conclusion and future work

The twenty-five-year-old idea (hand the network only the problem and the error, and let it find its own shape) now runs end to end: architecture, training hyper-parameters, and even the choice between a dense and a weight-sharing layer are discovered by evolution, and the search halts when the target error is met. On the hypothesis of Section 2, the 400-seed evidence supports H and rejects H_0 : the self-modifying search reached the target more often than random (paired $p \approx 0.02$) and, when both arrived, in fewer generations on the majority of runs (head-to-head 146 to 87, $p \approx 7 \times 10^{-5}$). The effect is modest in size but statistically clear, and confined to the time-to-target regime; we do not claim it transfers to the fixed-budget measure, where random remains a strong baseline. Three next steps remain. First, *recombination*: the engine now carries an optional crossover operator (`evolve -X`, the percentage of offspring made by crossing two elites) that splices two parents by a one-point join of their hidden-layer lists and blends their training hyper-parameters, the rest by the usual mutation. Whether adding this building-block

mixing to mutation reaches the target faster is a direct on/off time-to-target A/B, and is the immediate next experiment; the search so far is mutation-only (crossover off). Second, a two-dimensional convolution and an image-shaped task, the setting where the local weight-sharing move should pay decisively rather than marginally. Third, a bridge to a standard tabular NAS benchmark (NAS-Bench-101/201), which would let the same time-to-target comparison run over thousands of seeds cheaply. Each is one command away in the released code.

Reproducibility

Every result above is regenerated from [13] with a single command. The engine builds with `make` (a C99 compiler, no dependencies) into three programs: `smbpann` (the worker), `evolve` (the search and control), and `gentask` (the task generator). The convolutional gradient check and the other correctness tests run as `make ut` under AddressSanitizer and UBSan. `scripts/errortest.sh` runs the time-to-target experiment; its controlling variables are environment variables: `DIM`, `N`, `FREQ`, `NOISE` (the task); `TARGET` (the target error); `LMAX`, `WMAX` (the search space); `POP`, `GENS`, `ELITE`, `MUT`, `ADAPT` (the search); `EPOCHS` and `RUNS`; and an optional `FIXTASK` to hold the task fixed. A further `evolve` flag, `-X` (percent of offspring made by crossover), is 0 throughout this paper. For example, Table 2 is

```
DIM=8 N=300 FREQ=3 NOISE=2 TARGET=0.17 POP=10 GENS=30 EPOCHS=1000
LMAX=3 WMAX=16 MUT=2 ADAPT=1 ELITE=2 RUNS=400 scripts/errortest.sh
```

and `FIXTASK=1` switches to the fixed-task design. The per-task heterogeneity of Table 3 is `scripts/errortest_multi.sh` with the same variables plus `NTASKS` and `SEEDS` (the number of fixed tasks and of search seeds per task):

```
NTASKS=8 SEEDS=50 DIM=8 N=300 FREQ=3 NOISE=2 TARGET=0.17 POP=10 GENS=30
EPOCHS=1000 LMAX=3 WMAX=16 MUT=2 ADAPT=1 ELITE=2 scripts/errortest_multi.sh
```

Each run uses a fixed 32-bit xorshift generator, so the numbers are identical on re-running.

References

- [1] D. E. Rumelhart, G. E. Hinton, R. J. Williams. Learning representations by back-propagating errors. *Nature* 323, 1986.
- [2] V. Gavrilov. *Backpropagation Feed-Forward Neural Networks*. Seminar in Artificial Intelligence, Open University of Israel, 1997.
- [3] Y. LeCun, B. Boser, J. S. Denker, et al. Backpropagation Applied to Handwritten Zip Code Recognition. *Neural Computation* 1(4), 1989.
- [4] I. Rechenberg. *Evolutionstrategie*. 1973; H.-P. Schwefel, *Numerical Optimization of Computer Models*, 1981 (self-adaptive mutation in evolution strategies).
- [5] K. O. Stanley, R. Miikkulainen. Evolving Neural Networks through Augmenting Topologies. *Evolutionary Computation* 10(2), 2002.
- [6] J. Bergstra, Y. Bengio. Random Search for Hyper-Parameter Optimization. *JMLR* 13, 2012.
- [7] B. Zoph, Q. V. Le. Neural Architecture Search with Reinforcement Learning. *ICLR* 2017.
- [8] E. Real et al. Large-Scale Evolution of Image Classifiers. *ICML* 2017.
- [9] E. Real, A. Aggarwal, Y. Huang, Q. V. Le. Regularized Evolution for Image Classifier Architecture Search. *AAAI* 2019.
- [10] H. Liu, K. Simonyan, Y. Yang. DARTS: Differentiable Architecture Search. *ICLR* 2019.

- [11] L. Li, A. Talwalkar. Random Search and Reproducibility for Neural Architecture Search. *UAI* 2019.
- [12] T. Elsken, J. H. Metzen, F. Hutter. Neural Architecture Search: A Survey. *JMLR* 20, 2019.
- [13] V. Gavrilov. SMBPANN: a self-modifying backpropagation feed-forward neural network. Source code, <https://github.com/Anode1/SMBPANN>.